

Low-Level *Design*

Per-service internals — APIs, data models, algorithms, schemas, and sequence flows for Search, Recommendations, and Business Analysis.

DOCUMENT	LEVEL	AUDIENCE
LLD v1.0	Module / Class	Engineers

Public API

METHOD	ENDPOINT	PURPOSE
POST	/v1/search	Hybrid query → ranked results
POST	/v1/index	Ingest & embed a document
DELETE	/v1/index/{id}	Remove a document vector

Request / Response contract

```
# POST /v1/search
{ "query": "wireless noise cancelling headphones",
  "top_k": 20, "filters": { "category": "audio" },
  "alpha": 0.6 }           # vector vs BM25 weight
# → 200
{ "results": [ { "id", "score", "snippet" } ],
  "took_ms": 142 }
```

Retrieval algorithm

```
def search(q, top_k, alpha):
    qv = embed(q)           # 1. embed query
    vec = vectordb.ann(qv, k=200) # 2. dense ANN
    kw = bm25.search(q, k=200)   # 3. sparse
    cand = rrf_fuse(vec, kw, alpha) # 4. reciprocal-rank fusion
    return cross_encoder.rerank(q, cand)[:top_k] # 5. precise re-rank
```

Why RRF — reciprocal-rank fusion combines dense and sparse rankings without needing comparable score scales, then the cross-encoder refines only the top candidates to keep latency bounded.

Two-stage pipeline

```
def recommend(user_id, context, n=10):
    u = feature_store.get_online(user_id)           # <10ms
    # Stage 1 – candidate generation (millions → ~500)
    c1 = cf_model.knn(u.embedding, k=300)          # collaborative
    c2 = vectordb.ann(u.taste_vector, k=200)        # content (reused vectors)
    c3 = popular_by_segment(u.segment, k=100)       # fallback
    cand = dedupe(c1 + c2 + c3)
    # Stage 2 – ranking
    feats = build_features(u, cand, context)
    scores = ranker.predict(feats)                 # LightGBM / DNN
    return rank_and_filter(cand, scores, n)         # dedup, business rules
```

Ranking features

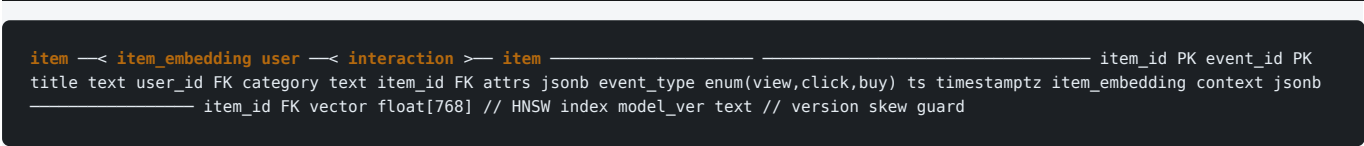
GROUP	EXAMPLES
User	recency, frequency, segment, historical CTR
Item	popularity, age, price tier, embedding sim
Context	device, time-of-day, session length, referrer
Cross	user-category affinity, prior interactions

Cold start — when `u.embedding` is null, the pipeline skips collaborative candidates and weights content + popularity, switching back as interaction events accumulate via the event stream.

Agent flow

1	user → service	Natural-language question received
2	service → vectordb	Retrieve relevant schema + metric definitions (RAG)
3	service → LLM	Prompt: question + schema context → candidate SQL
4	service → validator	Parse SQL, check tables/cols, enforce row-level filter
5	service → warehouse	Execute read-only query (LIMIT-guarded)
6	service → LLM	Summarise rows → narrative + chart spec
7	service → user	Answer + generated SQL (for transparency)

Guardrails — the LLM never touches the database directly. A validation layer restricts it to a whitelisted, read-only schema and injects tenant row-level security before any query runs.



STORE	HOLDS	ACCESS PATTERN
Vector DB	item_embedding	ANN by similarity
OLTP / Postgres	item, user	point lookups
Feature Store	derived user/item features	online <10ms / offline batch
Warehouse	interaction facts, aggregates	OLAP scans for BI

CONCERN	APPROACH
Caching	Redis for hot queries, online features, and LLM-answer memoisation keyed by normalised question
Async ingest	Embedding generation off the write path via Kafka consumer + Airflow batch backfill
Versioning	model_ver on every vector; re-embed pipeline triggered on model upgrade with shadow comparison
Retraining	Nightly offline feature materialisation; weekly ranker retrain with offline eval gates before promotion
Observability	OpenTelemetry traces span gateway → service → store; model metrics (CTR, NDCG, query latency) on dashboards
Failure modes	Search degrades to BM25; recommender to popularity; analysis returns SQL without summary if LLM unavailable